



A JIAN CAPABILITY

Jian for Applications

Bespoke AI agents for the line-of-business applications your revenue actually runs on — watched, traced, and worklisted 24×7.

Infrastructure monitoring stops at the server. Jian for Applications goes inside the app — the claims feed, the clearinghouse, the billing pipeline — and tells you the moment money stops moving.

LIVE TODAY

3

agents live on the ORX claims platform

\$180K

in billed-but-unsent claims surfaced

9,295

claims caught in a silent feed-freeze gap

Your RMM watches servers. Your SIEM watches security. Your backup tool watches backups. **None of them can see a claims feed silently freeze, or 1,615 service lines billed in your system but never transmitted to the payer.** That logic lives **inside the application** — so that's where the agent has to live.

THE BLIND SPOT

Generic monitoring stops at the app's front door

- ✗ A sync job exits "success" (exit 0) but copies zero rows — the dashboard stays green for months
- ✗ Lines added to a claim after its first batch are never sent — billed, never paid
- ✗ Rejections pile up in a queue no alert is watching
- ✗ The one person who understood it all was a contractor — now gone

THE FIX

A dedicated agent that knows your app's business logic

- ✓ Reads your app's own tables, queues & APIs — not just CPU and disk
- ✓ Knows what "healthy" looks like — and what a revenue leak looks like
- ✓ Traces any record end-to-end and produces the exact fix worklist
- ✓ Documents the tribal knowledge so it never walks out the door again

THE MODEL WATCH → WORKLIST → (OPTIONAL) OPERATE



1

Watch

Read-only agents poll your app's data flow on a tight cadence, detecting freezes, leaks & failures the moment they happen.



2

Worklist

Every finding becomes a branded alert + a precise, grouped worklist your team can act on — never just "something's wrong."



3

Operate

Optionally, our engineers operate, fix & modernize the application itself — the work a dev shop used to do.

WHAT MAKES IT DIFFERENT

Read-only & alert-only

Jian watches and traces; it never auto-resubmits a claim or mutates billing. The judgment call stays human — because a duplicate claim is real money.

Built on the MCP pattern

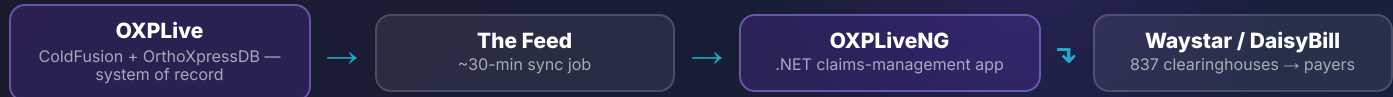
Each agent follows the SP → API → MCP reference architecture — the same one already running live in production for ORX's DaisyBill server.

Same engine, your logic

The proven Jian monitor-triage-alert engine — trusted across 57+ managed environments — taught the specifics of your one application.

Team Post-Op, Inc. dba OrthoKinetix — powered by OrthoXpress is a California orthopedic DMEPOS biller. Revenue depends on claims being built, transmitted, accepted & posted across **two payer worlds** — commercial / Medicare via Waystar, and CA workers'-comp via DaisyBill. When the prior developer was off-boarded, Technijian took the platform end-to-end and put Jian on it.

THE DATA FLOW JIAN WATCHES



The critical dependency: OXPLiveNG has no data of its own — it is entirely populated by the Feed. If the Feed stops, the app silently goes stale even while billing continues. **That single failure is the root of almost every “claims aren’t showing” report** — and exactly what Jian’s feed watchdog exists to catch.

THREE AGENTS ON ONE REVENUE CYCLE



OXPLive Feed Watchdog

READ-ONLY

Compares OXPLiveNG against the OrthoXpressDB system of record — job & claim recency, source-vs-destination lag, and the open backfill gap. Traces any job or claim across both databases.

Catch: a silent 95-day feed outage once froze ~9,295 claims out of the management app. The watchdog now flags a freeze within 15 minutes.



Waystar Clearinghouse Agent

READ-ONLY

Monitors 837 submission, 999 / 277CA / 835 responses, and the REST API. Detects the batching leak (lines added after first batch, never sent), groups rejections into fix buckets, and traces any claim end-to-end.

Catch: ~1,615 service lines / ~\$180K billed in the system but never transmitted — surfaced as a ready-to-resubmit worklist for the billing team.



DaisyBill 837 MCP Agent

READ-ONLY

Watches the live DaisyBill MCP server — bill-sync & EDI-837 recency, MCP + core-API health endpoints, and bill / payment counts — the CA workers'-comp half of the revenue cycle.

Status: 85 syncs / 82 EDI runs / 0 failures historically — now continuously health-checked for any >24h gap or 503.

When Technijian took the ORX platform end-to-end, Jian inherited a register of known failure modes — and now watches every one of them, continuously. Here is what each agent catches, how it tells you, and what lands on your desk.

THE FAILURE REGISTER WHAT EACH AGENT WATCHES FOR & HOW IT DETECTS IT

FAILURE MODE	SEV	HOW JIAN DETECTS IT
Feed freeze — the app silently goes stale	Revenue	Source-vs-destination lag; OXPLiveNG max date vs OrthoXpressDB
Batching leak — lines billed but never sent	Revenue	Unbatched-line scan + submission trace vs non-matched charges
Rejections aging — unworked denials	Reliability	Rejection summary grouped by exact fix bucket (payer, member ID...)
WC sync / EDI paused — DaisyBill gap	Reliability	Sync-run & EDI-837 recency + MCP / core-API health (503)

HOW JIAN COMMUNICATES — NEVER A SILENT FAILURE, NEVER A VAGUE ALARM

✉ Email

- Branded jian@ alerts, throttled
- The worklist, not just the symptom
- Routed to the right owner

💬 Teams

- Channel-per-issue @Jian bot
- Status & comments flow in-thread
- Self-service questions answered

📄 Client Portal

- Idempotent ticket per finding
- Shift-aware assignment
- Auto time entries on the work

WHAT YOU GET — EVERY ENGAGEMENT

- ✓ **24×7 continuous monitoring** of your application's data flow & health
- ✓ **End-to-end record tracing** on demand — any claim, any job
- ✓ **Monthly activity & health report**, branded
- ✓ **Actionable worklists** grouped by exact fix, not raw error dumps
- ✓ **A living failure-modes & maintenance handbook** for your app
- ✓ **Credentials secured in vault** — never in code, logs or tickets

The bottom line — every revenue-impacting failure on this platform now has a **watcher**, a **worklist**, and an **owner**, 24×7.

Three clean lines. You **land on the watch** — a low monthly fee per agent — and expand into full managed operations only as trust builds. Pricing is anchored to the revenue an app agent protects, not to headcount; one missed billing event dwarfs a year of watching.

★ THE RECURRING LINE

ONE-TIME

Build**\$8K–\$25K**

per application agent

Teach Jian your app: discovery, **SP** → **API** → **MCP** integration, the detection logic, and the operating runbook. Scoped to your app's complexity.

PER AGENT / MONTH

Run**\$250–\$500**/ agent /
mo

Continuous 24×7 watch, branded alerts, worklists & the monthly report. **Bundle a whole platform** — multiple agents on one application — for a single monthly figure.

OPTIONAL

Managed Ops**Retainer** / T&M

Our engineers **operate, fix & modernize** the application itself — the work a dev shop did — billed on your managed-services rate card.

WORKED
EXAMPLE

ORX claims platform — three agents (OXPLive Feed + Waystar + DaisyBill) bundled as “Jian for the ORX Claims Platform”, plus a one-time build and an optional operations retainer for the platform takeover.

\$750–\$1,500

/ month · 3-agent bundle

EVERY PLAN INCLUDES

- ✓ 24×7 read-only monitoring & tracing
- ✓ Grouped, ready-to-act worklists
- ✓ Monthly activity & health report
- ✓ Branded email + Teams + Client Portal alerts
- ✓ Living failure-modes & maintenance handbook
- ✓ Vault-secured credentials, least privilege

Put an agent on the app your revenue runs on.

Scoped in days, watching in weeks — on the same engine trusted across 57+ managed environments. Indicative bands; final pricing confirmed on scoping.

✉ sales@technijian.com

☎ (949) 379-8499

🌐 technijian.com